

Introduction au génie logiciel

Objectifs

- À la fin de ce chapitre, l'étudiant doit être capable de :
 - définir le génie logiciel ;
 - expliquer pourquoi le développement logiciel doit être organisé ;
 - distinguer logiciel, programme et système d'information ;
 - identifier les grandes étapes de réalisation d'un logiciel ;
 - comprendre les notions de cycle de vie, méthode, qualité et maintenance ;
 - reconnaître les principaux acteurs d'un projet logiciel

Introduction : pourquoi parler de génie logiciel ?

- **Constat de départ**

- Créer un logiciel ne consiste pas seulement à écrire du code.
- Lorsqu'un logiciel devient un peu grand, plusieurs difficultés apparaissent :
 - les besoins changent ;
 - plusieurs personnes travaillent ensemble ;
 - il faut éviter les erreurs ;
 - il faut livrer à temps ;
 - il faut maintenir le logiciel après sa mise en service.
- Le génie logiciel est né pour répondre à ces problèmes.

Exemple

- Prenons le cas d'une application de **gestion de bibliothèque**.
 - Au début, un étudiant peut écrire seul un petit programme qui :
 - enregistre des livres ;
 - ajoute des adhérents ;
 - suit les emprunts.
 - Mais si l'application doit ensuite :
 - gérer plusieurs bibliothèques ;
 - avoir une interface web ;
 - sécuriser les accès ;
 - produire des statistiques ;
 - permettre la réservation en ligne
 - alors le simple "codage direct" ne suffit plus.
Il faut une démarche organisée.

Crise du logiciel

- Constat du développement logiciel fin années 60 :
 - délais de livraison **non respectés**
 - budgets **non respectés**
 - **ne répond pas aux besoins** de l'utilisateur ou du client
 - **difficile** à utiliser, maintenir, et faire évoluer
- La **crise du logiciel** désigne l'ensemble des difficultés rencontrées dans le développement des logiciels à partir du moment où ceux-ci sont devenus plus gros, plus complexes et plus coûteux à produire.
- Elle est apparue lorsque les méthodes classiques de programmation ne suffisaient plus pour maîtriser correctement les projets informatiques.
- Autrement dit, les équipes arrivaient à programmer, mais avaient de plus en plus de mal à :
 - terminer les projets à temps ;
 - respecter les budgets ;
 - produire des logiciels fiables ;
 - faire évoluer les systèmes ;
 - maintenir le code dans la durée.

Crise du logiciel

- Dans les débuts de l'informatique, les programmes étaient relativement petits.
- Un seul développeur pouvait souvent comprendre tout le système.
- Mais avec l'évolution des besoins :
 - les applications sont devenues plus volumineuses ;
 - les organisations ont voulu automatiser davantage d'activités ;
 - plusieurs développeurs ont commencé à travailler sur un même projet ;
 - les utilisateurs sont devenus plus exigeants ;
 - les systèmes ont dû être maintenus pendant plusieurs années.
- Cette augmentation de complexité a rendu le développement beaucoup plus difficile.

Conséquences de la crise du logiciel

- La crise du logiciel a eu plusieurs conséquences importantes :
 - échec de nombreux projets ;
 - perte d'argent ;
 - insatisfaction des utilisateurs ;
 - abandon de certains systèmes ;
 - multiplication des bugs ;
 - logiciels difficiles à maintenir ;
 - manque de confiance dans les systèmes informatiques.
- **Exemple**
 - Une administration met en place un nouveau système de gestion,
 - mais les agents continuent à travailler manuellement parce que le logiciel est trop lent, incomplet ou peu fiable.

Pourquoi cette crise a conduit au génie logiciel

- La crise du logiciel a montré qu'il ne suffisait pas de savoir programmer.
- Il fallait aussi :
 - analyser les besoins avant de coder ;
 - planifier le travail ;
 - concevoir l'architecture ;
 - organiser les équipes ;
 - tester méthodiquement ;
 - documenter le système ;
 - prévoir la maintenance.
- C'est pour répondre à ces difficultés qu'est né le **Génie Logiciel**.

Pourquoi cette crise a conduit au génie logiciel

- Le génie logiciel apporte donc :
 - des méthodes ;
 - des modèles de cycle de vie ;
 - des règles de qualité ;
 - des techniques de conception ;
 - des outils de test ;
 - des pratiques de maintenance.

Définition du génie logiciel

- Plus qu' une branche de l'informatique
- C'est une discipline à part entière
- Donc regroupe plusieurs activités
 - Méthodologies (gestion des projets, conception des logiciels, ...)
 - Pratiques (programmation, interface matériel-logiciel, AGL, ...)
 - Qualité (test, vérification, ...)

Définition du génie logiciel

- Difficile pour un homme de s'attaquer à tous ces aspects
 - chacun doit se frayer un chemin
 - D'où la diversité des profils de formation (IUT, BTS, Ingénieur de Conception, MIAGE, ...)
- Mettre en relation plusieurs compétences pour produire le meilleur
 - Complémentarité dans la différence

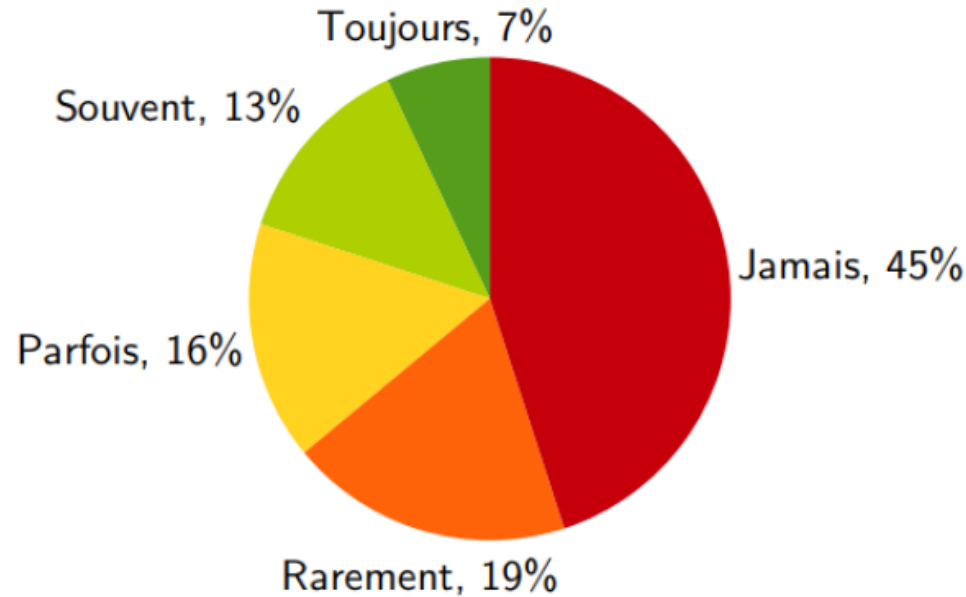
Définition du génie logiciel

- Le **génie logiciel** est l'ensemble des **méthodes, techniques, outils et bonnes pratiques** utilisés pour **concevoir, développer, tester, déployer et maintenir** des logiciels de qualité.
- Autrement dit, il s'agit d'une manière **rigoureuse et organisée** de produire des logiciels.
- Le génie logiciel applique au logiciel une logique proche de l'ingénierie :
 - on analyse avant de construire ;
 - on planifie ;
 - on conçoit ;
 - on contrôle la qualité ;
 - on maintient après livraison.

Exemple

- Pour construire une maison, on ne commence pas directement par poser des briques sans plan.
On commence par :
 - identifier le besoin ;
 - produire un plan ;
 - choisir les matériaux ;
 - organiser les travaux ;
 - contrôler la qualité.
- Pour un logiciel, c'est pareil :
 - besoin du client ;
 - spécifications ;
 - conception ;
 - développement ;
 - test ;
 - maintenance.

Utilisation des fonctionnalités implantées



Standish group, *Chaos Manifesto 2002*, 2002

« La **satisfaction** du client et la **valeur** du produit sont plus grandes lorsque les **fonctionnalités** livrées sont bien **moins nombreuses** que demandé et ne remplissent que les **besoins évidents**. »

Différence entre programme, logiciel et génie logiciel

- **Programme**

- Un **programme** est un ensemble d'instructions écrites dans un langage de programmation pour exécuter une tâche.
- Exemple : Un programme qui calcule la moyenne d'un étudiant.

- **Logiciel**

- Un **logiciel** est plus large qu'un programme.
- Il comprend :
 - les programmes ;
 - les données ;
 - la documentation ;
 - les procédures d'utilisation ;
 - parfois les scripts d'installation et de configuration.

- Exemple

- Un logiciel de paie contient :
 - l'application ;
 - la base de données ;
 - les interfaces ;
 - les manuels utilisateurs ;
 - les procédures de sauvegarde.

Différence entre programme, logiciel et génie logiciel

- **Génie logiciel**

- Le génie logiciel correspond à la **démarche de production et de gestion** de ce logiciel.

- **Résumé**

- **Programme** = code pour une tâche.
 - **Logiciel** = ensemble exploitable par l'utilisateur.
 - **Génie logiciel** = méthode pour produire correctement ce logiciel.

Pourquoi le génie logiciel est-il nécessaire ?

- **Gérer la complexité**

- Les logiciels modernes peuvent contenir :
 - beaucoup de fonctionnalités ;
 - de nombreux utilisateurs ;
 - plusieurs modules ;
 - des interactions avec d'autres systèmes.
- Une plateforme universitaire peut gérer :
 - les inscriptions ;
 - les notes ;
 - les emplois du temps ;
 - les paiements ;
 - les enseignants ;
 - les étudiants ;
 - les bibliothèques.
- Sans organisation, le projet devient vite ingérable.

Pourquoi le génie logiciel est-il nécessaire ?

- **Réduire les erreurs**

- Un logiciel mal conçu peut contenir :

- des bugs ;
 - des incohérences ;
 - des pertes de données ;
 - des failles de sécurité.

- Exemple

- Dans une application bancaire, une erreur sur le calcul du solde peut avoir de graves conséquences.

Pourquoi le génie logiciel est-il nécessaire ?

- **Respecter les délais et les coûts**

- Le génie logiciel permet :
 - d'estimer le travail ;
 - de répartir les tâches ;
 - de suivre l'avancement ;
 - de limiter les retards.

- **Faciliter la maintenance**

- Un logiciel vit longtemps après sa première version.
Il faut souvent :
 - corriger des erreurs ;
 - ajouter des fonctions ;
 - adapter le logiciel à un nouveau besoin.
- Une école demande après un an l'ajout d'un module de paiement mobile dans son application de scolarité.

Pourquoi le génie logiciel est-il nécessaire ?

- Assurer la qualité
 - Le logiciel doit être :
 - utile ;
 - fiable ;
 - compréhensible ;
 - sécurisé ;
 - maintenable. Etc...

Objectifs du génie logiciel

- **Produire un logiciel qui répond au besoin**
 - Un logiciel n'est pas bon seulement parce qu'il fonctionne techniquement. Il doit surtout résoudre le problème de l'utilisateur.
 - Exemple
 - Un logiciel de gestion hospitalière doit réellement permettre :
 - l'enregistrement des patients ;
 - la consultation des dossiers ;
 - la facturation ;
 - la planification des rendez-vous.

Objectifs du génie logiciel

- **Produire un logiciel de qualité**

- La qualité d'un logiciel se mesure par plusieurs critères :
 - fiabilité ;
 - performance ;
 - sécurité ;
 - facilité d'utilisation ;
 - facilité de maintenance.
 - etc...

Objectifs du génie logiciel

- **Produire dans de bonnes conditions**

- Le projet doit être mené :
 - dans le temps prévu ;
 - avec les ressources disponibles ;
 - avec une organisation claire.

- **Favoriser l'évolution du logiciel**

- Le logiciel doit pouvoir évoluer sans être entièrement réécrit.

Les caractéristiques d'un bon logiciel

- **Correction** : Le logiciel produit les résultats attendus.
 - Un programme de calcul de moyenne doit renvoyer la bonne moyenne
- **Fiabilité** : Le logiciel fonctionne sans tomber fréquemment en panne.
 - Une plateforme d'inscription en ligne doit rester stable pendant les périodes de forte affluence.
- **Efficacité**
 - Le logiciel utilise correctement les ressources : mémoire , processeur , réseau , stockage.
- **Maintenabilité** : Le logiciel peut être modifié facilement.
 - Ajouter un nouveau type d'utilisateur sans casser les modules existants.
- **Portabilité** : Le logiciel peut fonctionner sur différents environnements.
 - Une application accessible sur Windows, Linux et smartphone.
- **Réutilisabilité** : Certaines parties peuvent être réutilisées dans d'autres projets.
 - Un module d'authentification réutilisé dans plusieurs applications.
- **Sécurité** : Le logiciel protège les données, les accès et les opérations sensibles.

Les acteurs d'un projet logiciel

- Le client
 - C'est celui qui exprime le besoin ou finance le projet.
 - Une université demande une application de gestion académique.
- Les utilisateurs
 - Ce sont les personnes qui utiliseront le logiciel.
 - étudiants ; enseignants ; agents administratifs.
- L'analyste
 - Il étudie les besoins et formalise les exigences.
- Le concepteur
 - Il définit l'architecture et l'organisation du logiciel.
- Le programmeur
 - Il écrit le code source.
- Le testeur
 - Il vérifie que le logiciel fonctionne correctement.
- Le chef de projet
 - Il organise le travail de l'équipe :
 - planning ; coordination ; suivi ; communication.

Les acteurs d'un projet logiciel

- L'administrateur / mainteneur
 - Il assure le déploiement, le suivi technique et les évolutions.



cycle de vie du logiciel

- **Définition**

- Le **cycle de vie du logiciel** est l'ensemble des étapes suivies depuis l'idée initiale jusqu'à la mise hors service du logiciel.

- **Principales étapes**

- **a) Étude des besoins**

- On identifie : le problème ; les attentes ; les contraintes.

- **Exemple**

- Pour une application de pharmacie :
 - enregistrer les médicaments ;
 - gérer les ventes ;
 - alerter sur les ruptures de stock.

cycle de vie du logiciel

- **b) Analyse**
- On précise ce que le système doit faire.
- Exemples de questions :
 - Qui utilise le système ?
 - Quelles données faut-il stocker ?
 - Quelles opérations seront possibles ?
- **c) Conception**
- On prépare la solution technique :
 - architecture ;
 - base de données ;
 - interfaces ;
 - modules.
- **Exemple**
 - Créer les modules :
 - gestion des produits ;
 - gestion des clients ;
 - gestion des ventes ;
 - rapports.

cycle de vie du logiciel

- **d) Implémentation**

- C'est la phase de codage.

- **e) Tests**

- On vérifie que le logiciel respecte les besoins.

- **f) Déploiement**

- Le logiciel est installé et mis à disposition.

- **g) Maintenance**

- On corrige, améliore et adapte le logiciel.

cycle de vie du logiciel

- Linéaire (Mais pas bon)

